

Criando um projeto React + Typescript com Vite

Documentação oficial:

- <https://vite.dev/guide/> - Vite
- <https://react.dev/> - React
- <https://www.typescriptlang.org/> - TypeScript

1. Criando o projeto

É necessário executar o comando para iniciar a instalação:

npm create vite@latest . -- --template react-ts

O comando cria um projeto vite no diretório atual (sem criar outro diretório), e já o configura para utilização do React e do TypeScript. Caso prefira configurar no momento da instalação, basta executar o comando (npm create vite@latest).

Insira o nome do projeto, e então o Vite irá criar o template já com React e TypeScript

```
PS C:\Users\Jhean Bryan\Desktop\jhean\projetoReact> npm create vite@latest . -- --template react-ts

> npx
> create-vite . --template react-ts

◇ Package name:
  projetoreact

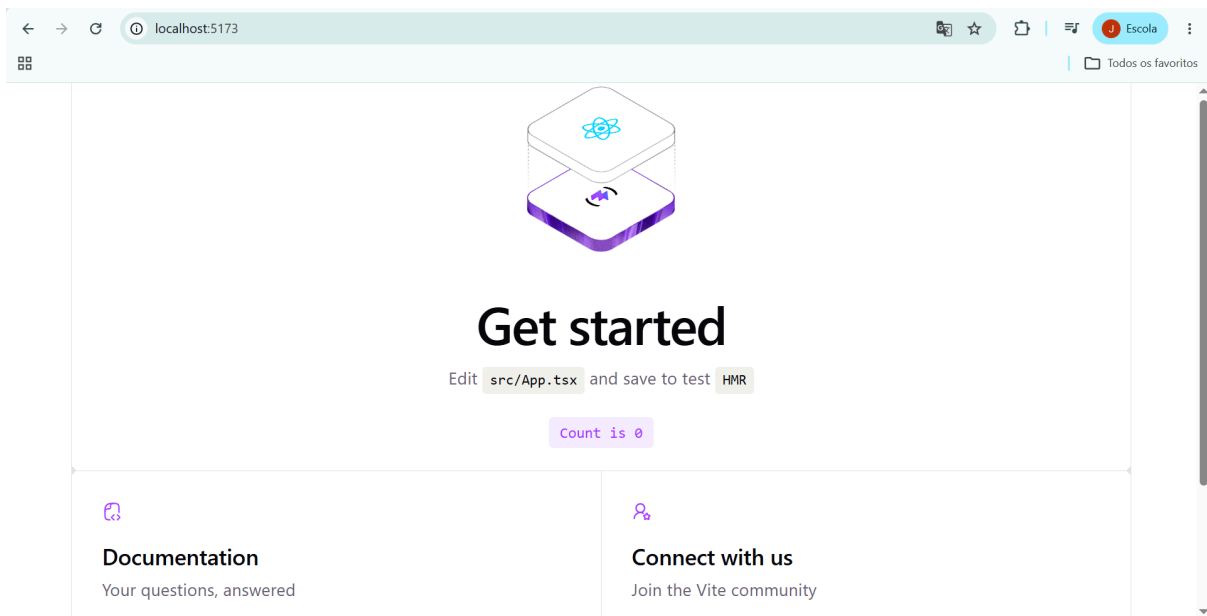
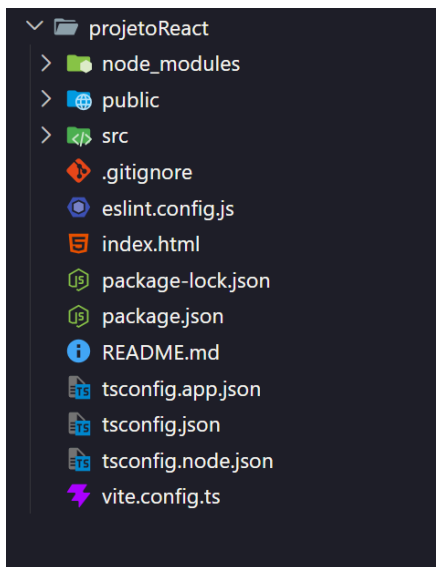
◇ Which linter to use?
  ESLint

◇ Install with npm and start now?
  Yes

◇ Scaffolding project in C:\Users\Jhean Bryan\Desktop\jhean\projetoReact...
```

```
VITE v8.2.1 ready in 2163 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

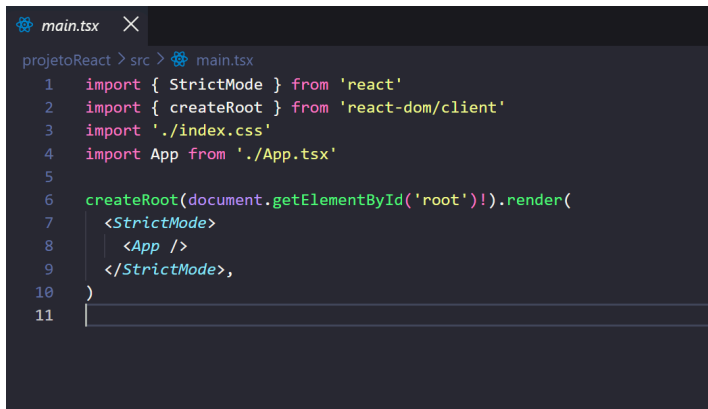


2. Limpando o template

Inicialmente, devemos 'limpar' este template. Podemos apagar alguns itens que não utilizaremos, como por exemplo, os arquivos de dentro da pasta **public**, e de **assets**:



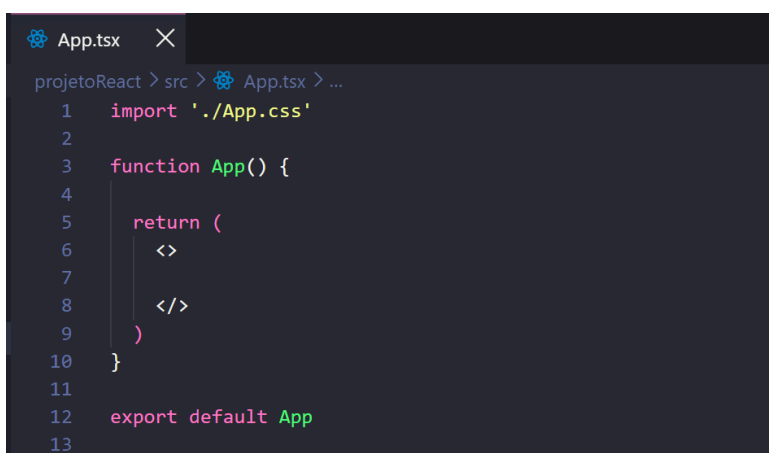
O arquivo **main.tsx** é o arquivo principal, pois através dele, os outros componentes serão renderizados:



```
main.tsx X
projetoReact > src > main.tsx
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.tsx'
5
6 createRoot(document.getElementById('root')!).render(
7   <StrictMode>
8     <App />
9   </StrictMode>,
10 )
11
```

O arquivo App contém o componente principal, podemos limpar todo o conteúdo de **App.tsx** e **App.css**, e **index.css**.

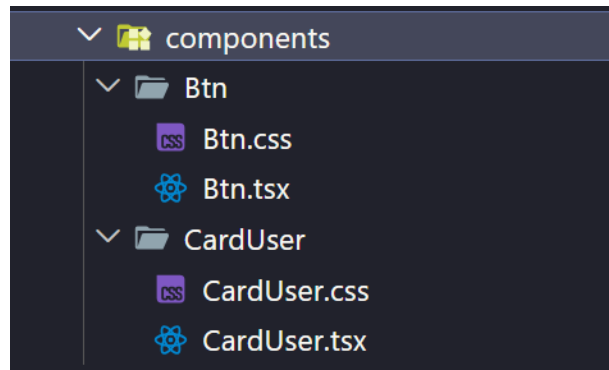
Dentro do **App.tsx**, colocaremos o 'esqueleto' base para nosso componente. Devemos importar o arquivo css, e criar uma função com o nome do componente, e também exportar essa função para, para ela estar disponível como componente (a exportação pode ficar ao final do código ou junto com a declaração da função, mas o mais comum é ao final do código). Por se tratar de uma função, devemos retornar algo, nesse caso, retornar o componente que queremos que esteja visível (para isso devemos usar o return dentro da função). Outra prática comum no projeto react, é o uso da tag coringa (react fragment, ou fragmento) `<></>`, que é basicamente uma tag que nos permite agrupar elementos sem criar nós da DOM, é uma boa prática muito utilizada.



```
App.tsx X
projetoReact > src > App.tsx > ...
1 import './App.css'
2
3 function App() {
4
5   return (
6     <>
7
8     </>
9   )
10 }
11
12 export default App
13
```

3. Criando componentes

Agora que já temos nosso projeto react limpo, iremos começar a criar nossos componentes. Dentro da pasta **src**, iremos criar uma pasta **components**, com mais duas pastas e dois arquivos em cada: No diretório Btn, criaremos dois arquivos com o mesmo nome, um para Css e outro tsx. Da mesma forma para o componente CardUser.

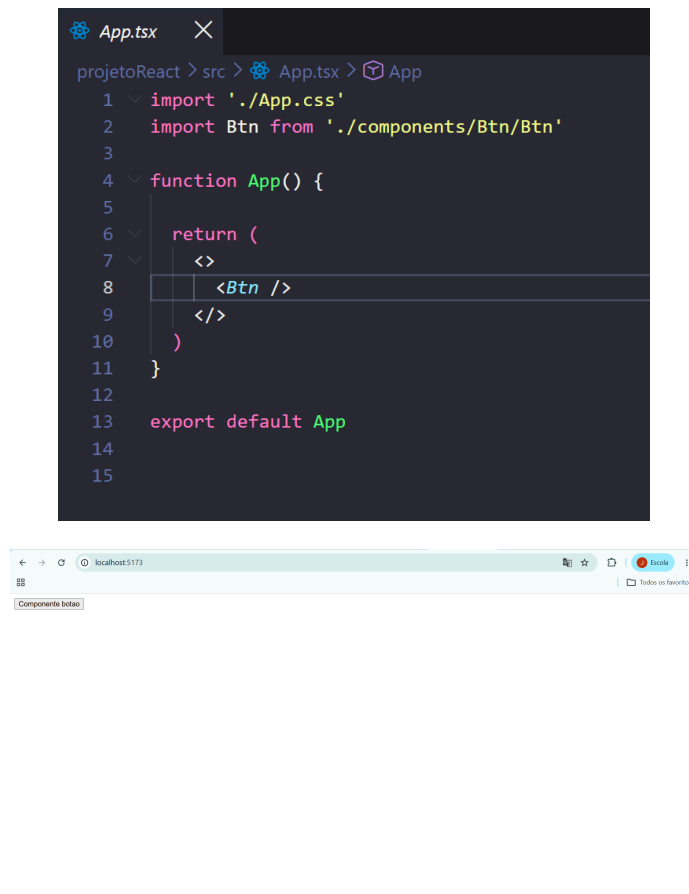


Neste exemplo, iremos criar um componente botão, e um card de usuário, para exibir algumas informações.

Agora podemos então criar nosso componente Btn, que será um botão:

```
Btn.tsx
projetoReact > src > components > Btn > Btn.tsx > ...
1  import './Btn.css'
2
3  function Btn() {
4
5      return (
6          <button>
7              Componente botao
8          </button>
9      )
10 }
11
12 export default Btn
13
```

Para exibir esse componente na tela, precisamos adicioná-lo no App.tsx. Devemos adicionar como uma tag de auto fechamento:



Agora que já criamos o componente `Btn`, vamos torná-lo **reutilizável**. Em React, um componente fica muito mais útil quando podemos personalizá-lo, por exemplo mudando o texto e a cor sem precisar criar vários botões diferentes.

```
1 import './Btn.css'
2
3 type BtnProps = {
4   texto: string;
5   corDeFundo: string
6 }
7
8 function Btn( {texto, corDeFundo }: BtnProps ) {
9
10  return (
11    <>
12      <button
13        className='btn'
14        style={{backgroundColor: corDeFundo}}
15      >
16        {texto}
17      </button>
18    </>
19  )
20 }
21
22
23 export default Btn;
```

Como estamos utilizando TypeScript, precisamos tipar os dados na linha 3, onde definimos um `type` chamado `BtnProps` que declara que as props `texto` e `corDeFundo` devem ser obrigatoriamente strings. Isso garante que quem usar o componente passe exatamente esses valores no formato correto, e o TypeScript avisa em tempo de desenvolvimento se houver erro. Na linha 6, desestruturamos as props recebidas e aplicamos o tipo `BtnProps`. Em seguida, no retorno do componente, renderizamos um botão com a classe `btn`, aplicamos a cor de fundo dinamicamente via `style` usando a prop `corDeFundo`, e exibimos o texto com `{texto}`. Por fim, exportamos o componente com `export default` para ser reutilizado em outros arquivos.

No arquivo de css do Btn, iremos passar apenas a estrutura base, pois a cor e texto será informada no momento da criação do componente:

```
1  .btn{
2      border-radius: 5px;
3      border: 1px solid #000;
4      font-size: 14px;
5      width: 80px;
6      height: 30px;
7      margin-inline: 10px;
8  }
```

Para o componente `CardUser`, seguimos a mesma lógica de tipagem do exemplo anterior. Na linha 3, definimos o tipo `CardUserProps` com três propriedades obrigatórias do tipo string: `nome`, `cargo` e `foto`. Isso garante que o componente sempre receba esses dados no formato correto. Na linha 8, desestruturamos as props e aplicamos o tipo. No retorno do componente, renderizamos um `div` com a classe `card-user`, contendo uma imagem que recebe a URL via prop `foto` e o texto alternativo com o `nome`, um título `h2` exibindo o `nome` e um `span` mostrando o `cargo`. Por fim, exportamos o componente com `export default` para reutilização. A

grande vantagem aqui é que o TypeScript nos protege contra erros, como esquecer de passar alguma prop ou passar um tipo incorreto, tornando o código mais seguro e previsível.

```
1  import './CardUser.css';
2
3  type CardUserProps = {
4    nome: string;
5    cargo: string;
6    foto: string;
7  }
8
9  function CardUser({nome, cargo, foto}:CardUserProps) {
10
11    return(
12      <>
13        <div className="card-user">
14          <img src={foto} alt={nome} />
15          <h2>{nome}</h2>
16          <span>{cargo}</span>
17        </div>
18      </>
19    )
20  }
21
22  export default CardUser;
```

Para o css do CardUser iremos apenas estilizar brevemente:

```

1  .card-user{
2      width: 220px;
3      padding: 16px;
4      border: 1px solid #000;
5      border-radius: 10px;
6      text-align: center;
7      background-color: #dcdcdc;
8      margin-top: 20px;
9  }
10
11  .card-user img{
12      width: 150px;
13      height: 150px;
14      border-radius: 50%;
15      object-fit: cover;
16  }
17
18  .card-user h2{
19      font-size: 38px;
20  }
21
22  .card-user span {
23      font-size: 24px;
24  }

```

No arquivo `App.tsx`, estamos importando e utilizando os componentes que criamos anteriormente. Na linha 2, importamos o componente `Btn` do caminho `./components/Btn/Btn`, e na linha 3 importamos o `CardUser` de `./components/CardUser/CardUser`. Dentro da função `App`, no retorno do JSX, usamos o componente `Btn` passando as props `texto='Salvar'` e `corDeFundo='green'`, que são exatamente as strings esperadas pelo tipo definido no componente. Em seguida, usamos o `CardUser` passando `nome='João'`, `cargo='Gerente'` e a foto com uma URL de uma imagem, todas como strings. Perceba que o TypeScript nos ajuda aqui também: se tentássemos passar um número no lugar de uma string ou esquecêssemos alguma prop, o editor já apontaria o erro automaticamente. Por fim, exportamos o `App` com `export default` para ser usado como ponto de entrada da aplicação. A estrutura fica limpa e organizada, com componentes reutilizáveis e fortemente tipados. Podendo adicionar mais componentes de acordo com a quantidade que precisaremos;

```
1  import './App.css'
2  import Btn from './components/Btn/Btn'
3  import CardUser from './components/CardUser/CardUser'
4
5  function App() {
6
7    return (
8      <>
9        <Btn texto='Salvar' corDeFundo='green' />
10
11        <CardUser
12          nome='Joao'
13          cargo='Gerente'
14          foto='https://images.unsplash.com/photo-1508921912186-1d1a45ebb3c1?q=80&w=687&
            auto=format&fit=crop&ixlib=rb-4.1.0&
            ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVuZDB8fHx8fA%3D%3D'
15        />
16      </>
17    )
18  }
19
20  export default App
```